Offline-First Personal Expense Manager

Mobile database case study using SQLite (Room)

Shubham Mishra (24BYB0117)
Anushka Jaiswal (24BYB0121)

Problem statement

- Expenses are logged on the move: buses, markets, low-signal areas
- Server-dependent apps hang or lose entries without a connection
- Data is personal and small: thousands of rows per year, one user
- A full client-server database per user is unnecessary overhead
- Question: how can a phone store, query and summarise expenses reliably, offline?

Objectives

- Study embedded mobile databases: SQLite and Android Room
- Design a normalised schema: category, expense, budget
- Build a working prototype: add, edit, delete, search, monthly reports
- Compare with client-server relational (PostgreSQL/MySQL) and cloud (Firestore)
- Demonstrate integrity, atomicity and measured performance

Selected technology: SQLite + Room

SQLite

Serverless, single-file SQL engine running inside the app. ACID transactions, indexes, constraints, triggers. Built into Android and iOS.

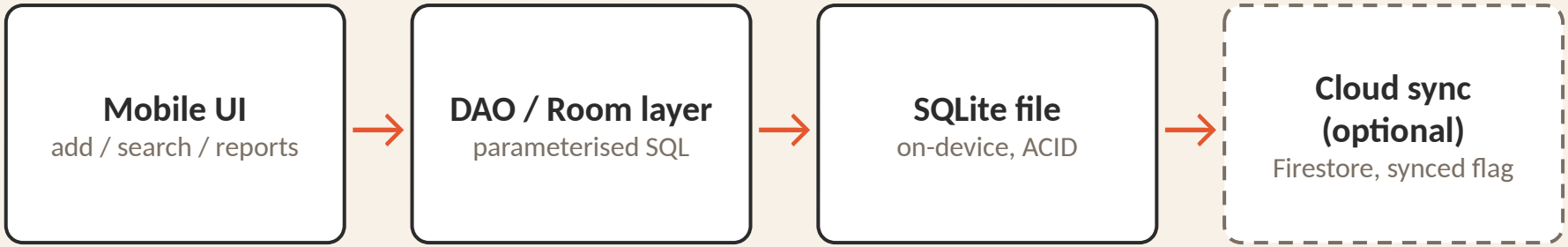
Room

Android layer over SQLite: entities, DAOs, compile-time query checks, observable queries that refresh the UI.

Why it fits

Works offline, in-process latency, zero administration, data isolated per device.

Architecture: local-first



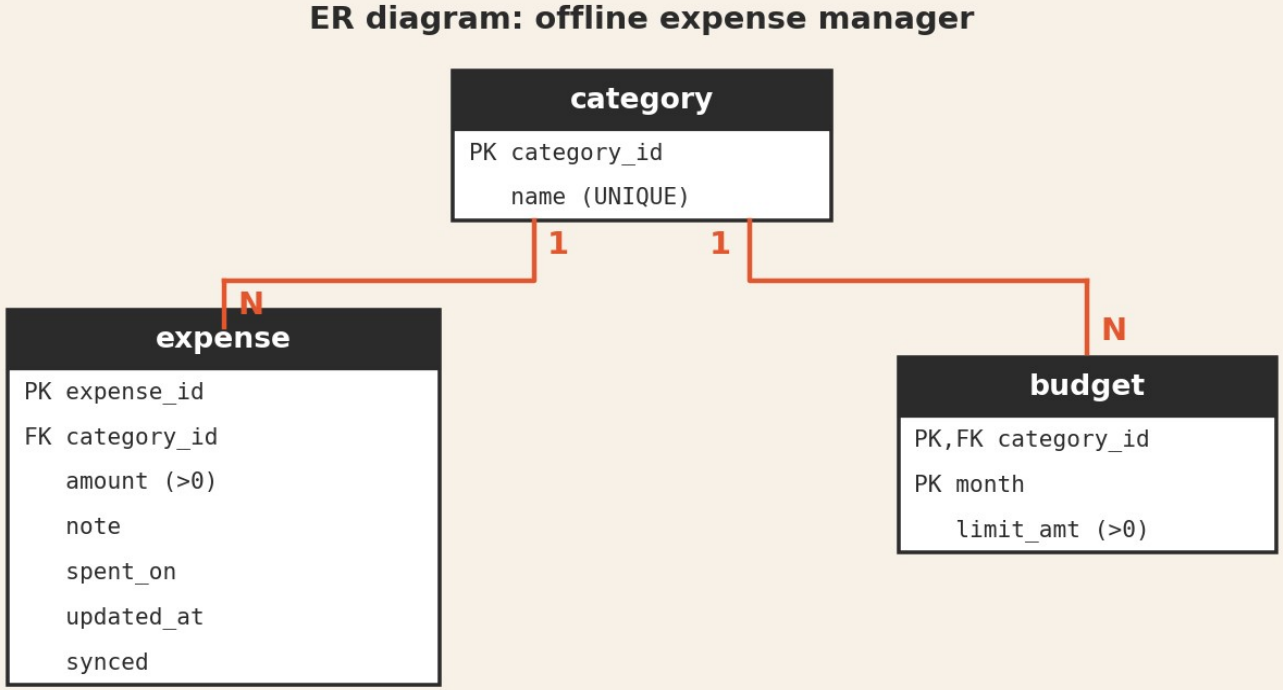
Every read and write hits SQLite first, so the app works with no network. Cloud sync is an add-on, not a dependency.

Database design

```
category(category_id PK, name UNIQUE)
expense(expense_id PK, amount CHECK>0, category_id FK, note,
        spent_on, updated_at, synced)
budget(category_id FK, month, limit_amt)  PK(category_id, month)
Indexes: expense(spent_on), expense(category_id)
```

- Normalised: categories stored once, referenced by key
- Constraints reject bad data inside the database itself
- synced flag marks rows waiting for cloud upload

ER diagram


ER diagram: offline expense manager
category
PK category_id
name (UNIQUE)
1
1
N
N
expense
PK expense_id
FK category_id
amount (>0)
note
spent_on
updated_at
synced
budget
PK,FK category_id
PK month
limit_amt (>0)

Implementation: key queries

Q1 Monthly total per category

```
SELECT c.name, SUM(e.amount) FROM expense e
JOIN category c USING(category_id)
WHERE strftime('%Y-%m', e.spent_on)='2026-10' GROUP BY c.name;
```

Q2 Categories over budget

```
SELECT c.name, SUM(e.amount) AS spent, b.limit_amt
FROM expense e JOIN category c USING(category_id)
JOIN budget b ON b.category_id=e.category_id
  AND b.month=strftime('%Y-%m', e.spent_on)
WHERE strftime('%Y-%m', e.spent_on)='2026-10'
GROUP BY c.name, b.limit_amt HAVING SUM(e.amount) > b.limit_amt;
```

Q3 Search by note / category

```
... WHERE COALESCE(e.note,'') LIKE '%text%' AND c.name = 'Food';
```

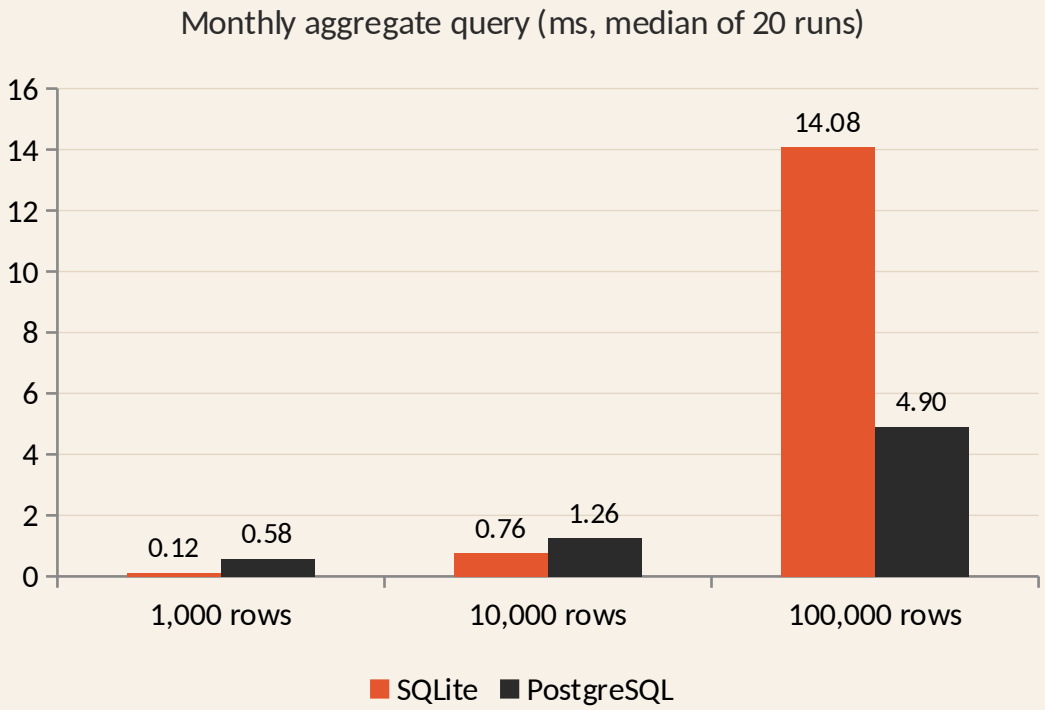
Q4 Pending sync

```
SELECT COUNT(*) FROM expense WHERE synced = 0;
```

Prototype results (150 rows)

Check	Result
Create DB, insert, update, delete with no network	Works; no server involved
Q1 October totals	Rent 5,500; Books 5,302; Health 2,177 …
Q2 over budget (Oct)	Books, Health, Entertainment
Negative amount / unknown category / bad foreign key	All rejected by constraints
Failed two-step transaction	Rolled back: 153 rows before, 153 after

SQLite vs PostgreSQL, measured

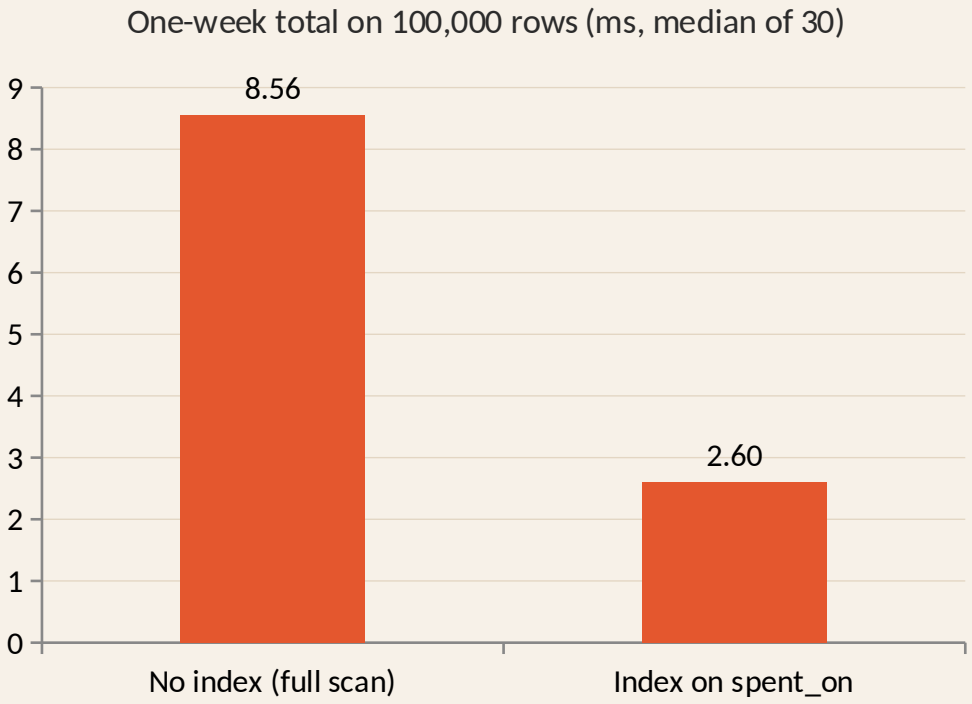


- SQLite faster on small tables; PostgreSQL about 2.9x faster at 100,000 rows
- A personal app holds thousands of rows, where SQLite wins or ties
- PostgreSQL on localhost; a phone would add network delay

Head-to-head results

Test	SQLite	PostgreSQL
Lookup by id	0.0061 ms	0.0784 ms (13x slower)
Lookups at 1k / 10k / 100k rows	0.008 / 0.006 / 0.006 ms	0.080 / 0.057 / 0.101 ms
Insert and commit	0.990 ms	1.415 ms
Bulk insert, 100,000 rows	395 ms	10,310 ms (row-by-row driver)
Server unavailable	Still reads and writes	Connection refused
4 concurrent writers x 200	1,179 ms	535 ms (2.2x faster)
Setup	Library, one 136 KB file	Server, user, access, backups

Technical insight: effect of the index



- EXPLAIN QUERY PLAN without index: SCAN expense
- With index: SEARCH expense USING INDEX idx_expense_date
- About 3.3x faster here; the gap grows with table size

Offline-to-cloud sync (simulated)

Step	Observed result
1 Offline: 23 entries saved	Pending sync = 23, cloud rows = 0
2 Still offline: add 1 more	Pending sync = 24
3 Connection returns: push	Pushed 24, pending 0, cloud rows 24
4 Conflict: older local edit vs newer cloud edit	Last-write-wins: local edit skipped, cloud value kept

A second SQLite file stands in for the cloud database; the same logic maps onto Firestore.

Comparison with other approaches

Aspect	SQLite / Room	PostgreSQL / MySQL	Cloud Firestore
Architecture	In-process file	Separate server	Managed cloud, documents
Offline use	Full	None by default	Cache supported
Latency	Local call	Network round trip	Network + cache
Administration	None	Install, secure, back up	Rules and project setup
Joins / SQL	Full SQL	Full SQL	Limited, denormalised
Many writers	Single writer	High concurrency	High concurrency
Cost	Free	Hosting	Pay per use

Advantages and limitations

Advantages

- Works fully offline
- Instant, in-process queries
- No server cost or administration
- ACID, constraints, full SQL
- Data stays on the device

Limitations

- Single-writer concurrency
- No built-in sync, auth or backup
- Data lost with device if not backed up
- Dynamic typing, dates as text
- Not for large shared workloads

Conclusion

- For small, personal, often-offline data, an embedded database beats client-server
- SQLite gives full relational integrity and ACID with zero setup
- Measured: SQLite 10-17x faster on lookups and works with no server; PostgreSQL 2.9x faster on a 100,000-row aggregate and 2.2x with concurrent writers
- Weak points (sync, multi-user, backup) are solved by a hybrid: SQLite as source of truth, cloud for sync
- Future work: Room/Kotlin UI, Firestore sync with last-write-wins conflict handling

References

- SQLite documentation and 'Appropriate Uses For SQLite', sqlite.org
- Android Developers: Save data in a local database using Room
- Firebase documentation: Cloud Firestore
- PostgreSQL documentation, postgresql.org
- Silberschatz, Korth, Sudarshan: Database System Concepts, McGraw-Hill